Block Different Shape Sorting

1. Function Description

After the program runs, the robotic arm will move to its initial position, then place color blocks of the same color but different shapes within the camera's view. Press keys to select the color of the blocks, then press the spacebar to grasp the shape blocks set by the program. The program defaults to grasping cylinders.

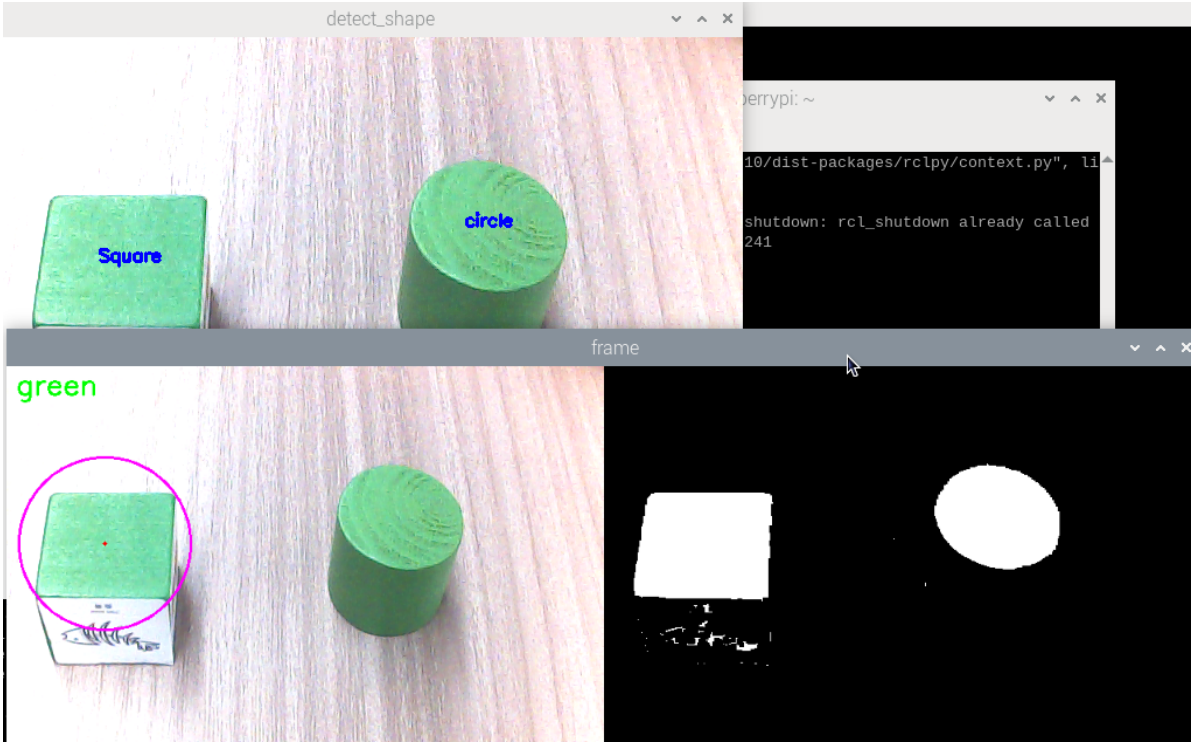
2. Function Startup

After entering the docker container, enter the following commands in the docker terminal:

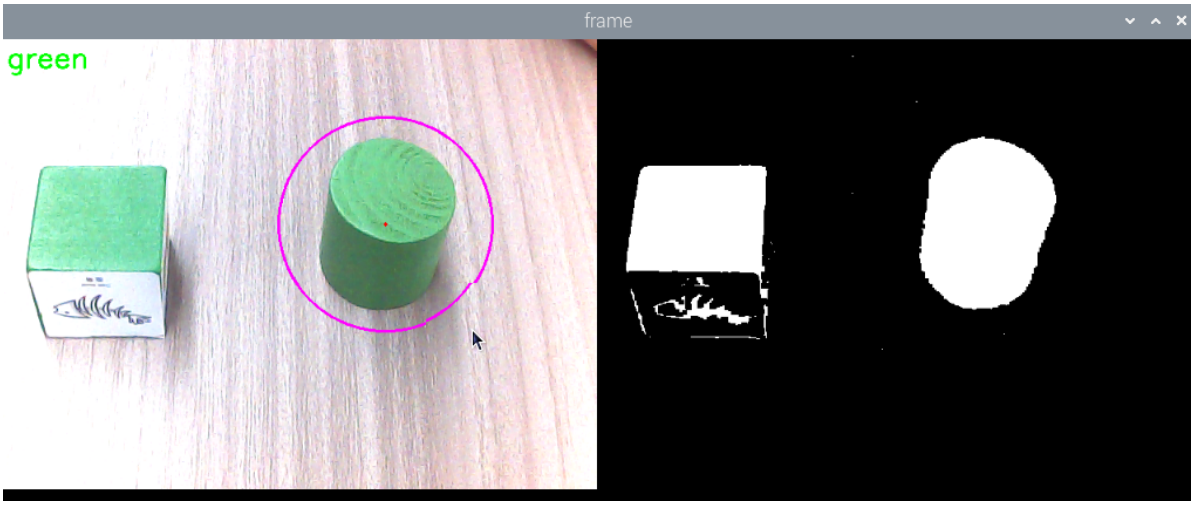
```
# Start USB camera node and inverse kinematics node
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start color block shape recognition node
ros2 run dofbot_sorting shape_sorting
# Start grasping node
ros2 run dofbot_sorting grasp
```

After successfully running the program, the robotic arm will move to its initial position, then place color blocks of the same color but different shapes within the camera's view as shown in the figure below.

Then, according to the block color, press r/g/b/y to perform block color recognition. If the binary image only shows the target color blocks, press the spacebar to start sorting as shown in the figure below:

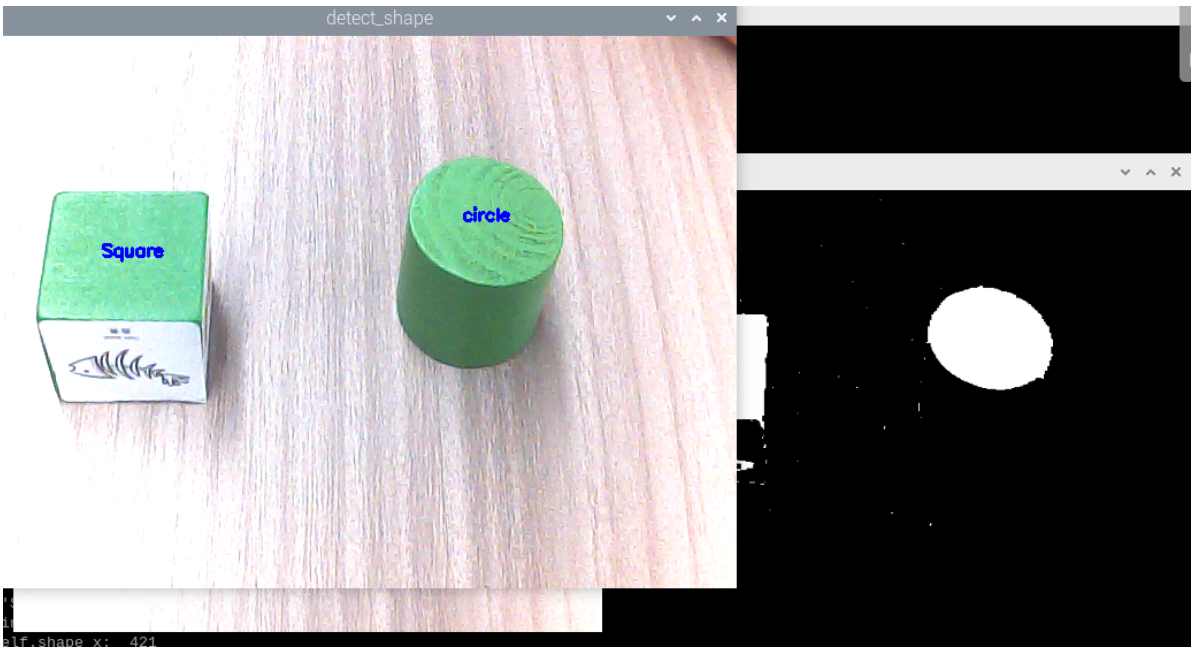


detect_shape will display the shape of the block top, which is used to determine what shape it is. Currently, it can only recognize rectangles corresponding to rectangular prisms, squares corresponding to cubes, and circles corresponding to cylinders. Press the spacebar, and the robotic arm will grasp the cylinder (program default) and place it at the designated position. If the binary image cannot recognize the top shape of the blocks or the top recognition effect is not good, as shown in the figure below:



Then you need to press c and use the mouse to select the top area of a certain block again for color calibration, as shown in the figure below:

Release the mouse to complete color calibration. The figure below will appear, and the top shape of each block is correctly recognized. You can press the spacebar to perform the grasping and placing action:



3. Code Analysis

3.1. shape_sorting

After entering the docker container, the source code path is located at:

```
/root/dofbot_ws/src/dofbot_sorting/dofbot_sorting/shape_sorting.py
```

Import necessary libraries:

```
import cv2
import os
import numpy as np
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2 as cv
# Defined color recognition library, source code location is at
/root/dofbot_ws/src/dofbot_sorting/dofbot_sorting/color_common.py
from dofbot_sorting.color_common import *
from dofbot_interface.msg import *
from std_msgs.msg import Bool,Int16
import time
from rclpy.node import Node
import rclpy
import threading
from Arm_Lib import Arm_Device
```

Create robotic arm control object and control the robotic arm to move to initial position:

```
self.Arm = Arm_Device()
self.init_joints = [90, 120, 0, 0, 90, 0]
self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
```

Create color recognition object:

```
self.color = color_detect()
```

Target shape for sorting:

```
self.target_shape = "circle"
```

Load HSV file paths for red, green, blue, and yellow colors:

```
package_pwd = "/root/dofbot_ws/src/dofbot_sorting/dofbot_sorting/"
self.red_hsv_text = os.path.join(package_pwd, 'red_colorHSV.text')
self.green_hsv_text = os.path.join(package_pwd, 'green_colorHSV.text')
self.blue_hsv_text = os.path.join(package_pwd, 'blue_colorHSV.text')
self.yellow_hsv_text = os.path.join(package_pwd, 'yellow_colorHSV.text')
#HSV content is as follows (using blue_colorHSV.text as example):
#55, 163, 235, 125, 253, 255
#Represent Hmin, Smin, Vmin, Hmax, Smax, Vmax respectively
```

Create publishers and subscribers:

```
# Create subscriber to subscribe to grasp completion topic messages
self.sub_grasp_status =
self.create_subscription(Bool,"grasp_done",self.get_graspStatusCallBack,100)
# Create subscriber to subscribe to image topic messages
self.subscription =
self.create_subscription(Image,'/image_raw',self.ImageCallback,1)
# Create publisher to publish recognized position information topic messages
self.pos_info_pub = self.create_publisher(AprilTagInfo,"PosInfo",1)
# Create publisher to publish calculated joint 5 angle reference value topic
messages
self.pub_joint5 = self.create_publisher(Int16, "set_joint5", 10)
```

Grasp completion topic callback function get_graspStatusCallBack:

```
# Process grasp completion topic messages, when receiving True, it means the
current sorting is completed,
# change self.pub_pos_flag to indicate that the next position information can be
published,
# and the completion flag is set to True
def get_graspStatusCallBack(self,msg):
    if msg.data == True:
        time.sleep(2)
        self.pub_pos_flag = True
        self.grasp_done = True
```

Image topic callback function ImageCallback:

```
# The first half of the color recognition steps are consistent with color block
recognition, both call the object_follow function of the created self.color
object. You can refer to the code analysis section in [Block Color Sorting] for
details, which will not be repeated here. This mainly analyzes how to perform
shape recognition based on binary images:
if len(binary) != 0 :
    img = binary.copy()
    # Extract contours from binary image
    contours, _ = cv2.findContours(img, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    detected_shapes = []
    # Traverse contour list
    for cnt in contours:
        # Calculate contour area
        area = cv2.contourArea(cnt)
        # If area is greater than 10000 (purpose is to filter out some impurity
points) and the previous grasping and sorting is completed, it means the next
block position information can be published
        if area > 10000 and self.grasp_done == True :
            #print("area: ",area)
            # Calculate contour perimeter
            perimeter = cv2.arcLength(cnt, closed=True)
            epsilon = 0.04 * perimeter
            # Perform polygon fitting, parameters are contour, approximation
accuracy and closed=True, indicating the polygon is closed
            approx = cv2.approxPolyDP(cnt, epsilon, closed=True)
            # Calculate number of polygon vertices
            num_vertices = len(approx)
            # Calculate contour circularity
```

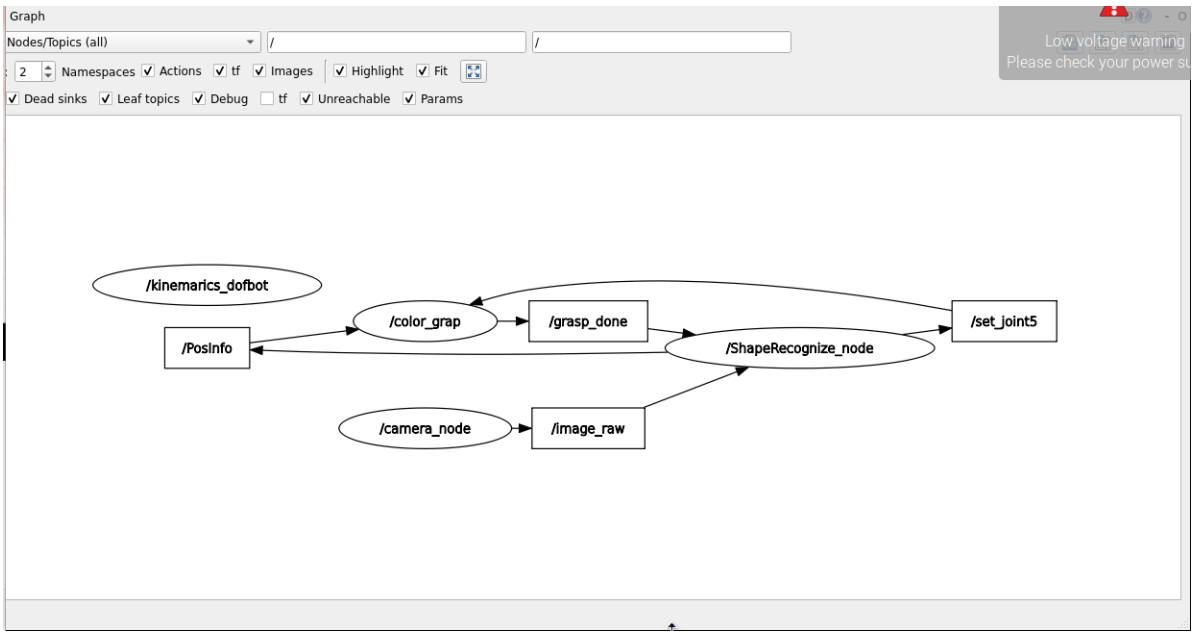
```
            circularity = (4 * np.pi * area) / (perimeter ** 2) if perimeter > 0
else 0
            #print("circularity: ",circularity)
            shape = "Unknown shape"
            # The circularity of a perfect circle is 1. Here, if circularity is
greater than 0.75, the polygon is considered circular
            if circularity > 0.75:
                shape = "circle"
            # If the number of vertices equals 4, it might be a square or
rectangle
            elif num_vertices == 4:
                # Calculate contour minimum bounding rectangle, return rectangle
position and size
                x, y, w, h = cv2.boundingRect(approx)
                # Calculate aspect ratio. If the ratio is between 0.95 and 1.4,
the length and width can be considered equal, indicating it's a square, otherwise
it's a rectangle
                aspect_ratio = w / float(h)
                if 0.95 <= aspect_ratio <= 1.4:
                    shape = "Square"
                else:
                    shape = "Rectangle"
            detected_shapes.append(shape)
            # Extract core features of contour
            M = cv2.moments(cnt)
            if M["m00"] != 0:
                # Get center point coordinates
                cX = int(M["m10"] / M["m00"])
                cY = int(M["m01"] / M["m00"])
                # If the currently recognized shape is the target shape (program
default is circle)
                if shape == self.target_shape:
                    print("Find the target shape block")
                    self.shape_x =  cX
                    self.shape_y =  cY
                    print("self.shape_x: ",self.shape_x)
                    print("self.shape_y: ",self.shape_y)
                cv2.putText(result_image, shape, (cX - 20, cY),
cv2.FONT_HERSHEY_SIMPLEX,0.5, (255, 0, 0), 2)

    if self.shape_x!=0 and self.shape_y!=0:
        print("..................................................")
        cx = int(self.shape_x)
        cy = int(self.shape_y)
        # Based on center point coordinates, convert coordinates to normalized
coefficients a and b. a is x-axis offset coefficient, b is y-axis offset
coefficient. The calculated ba values are the center point coordinates in the
world coordinate system, which will eventually be sent to the sorting grasp
node.
        (a, b) = (round(((320 - cx) / 4000), 5), round(((480 - cy) / 3000) *
0.8+0.15, 5))
        # Create position information topic message object
        pos = AprilTagInfo()
        pos.id = self.target_color
        pos.x = float(b)
        pos.y = float(a)
        pos.z = 0.03
        if self.pub_pos_flag == True:
```

```
            print("--------------------")
            self.pub_pos_flag = False
            # Publish position topic message
            self.pos_info_pub.publish(pos)
            print("*******************")
            self.shape_x = 0
            self.shape_y = 0
            self.grasp_done = False
```

3.2. grasp

Already analyzed in [13.3D Sorting and Grasping Course]-[1. AprilTag ID Sorting]-[3. Code Analysis], you can refer to the content there.

4. Node Communication Diagram

After entering the docker container, enter the following commands in the docker terminal:

```
ros2 run rqt_graph rqt_graph
```